SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 Total Marks:40

Code of Conduct:

I **_N.Umasri (192424240)_** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.Umasri

Criteria	**Understanding the case 2.5**	**Application of Concepts 2.5**	**Analysis 2**	**Solution Design 2**	**Clarity 1**	**Total (10)**
Q1						
Q2						
Q3						
Q4						

Faculty Signature

Case Study 1: Real-Time Video Conferencing Performance

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.

The problem is mainly related to the **transport protocol choice and its interaction with the application layer**. For real-time video conferencing, **UDP is generally preferred over TCP** because voice and video require low latency and continuous delivery.

The network is experiencing:

- 8% packet loss
- 180 ms jitter
- Fluctuating bandwidth utilization
- Voice distortion
- Frozen video frames
- Delayed communication

If TCP is being used for real-time media, packet loss causes retransmissions and head-of-line blocking. TCP waits for lost packets to be retransmitted before delivering subsequent data, which increases delay.

However, the application layer also plays an important role. Excessive buffering may compensate for jitter but can introduce additional delay. Therefore, the issue is best considered a **combination of transport protocol behavior and application-level buffering**, with the transport choice being the major concern.

2. Justify your answer based on the communication requirements of real-time multimedia applications.

Real-time multimedia applications have different requirements from ordinary web applications.

Video conferencing requires:

1. **Low latency** – Voice and video should reach the receiver quickly.

2. **Low jitter** – Packets should arrive at relatively consistent intervals.
3. **Continuous playback** – A small amount of missing data is often preferable to waiting for retransmission.
4. **Real-time delivery** – Old packets have limited value after their playback time has passed.
5. **Adaptive bandwidth usage** – The application should adjust video quality according to network conditions.

TCP provides reliable, ordered delivery, but retransmissions can cause significant delays. For example, if a video packet is lost, retransmitting it after several hundred milliseconds may be useless because the video frame may already have been displayed.

UDP does not retransmit lost packets automatically and therefore avoids much of this delay. Modern conferencing applications commonly use **UDP with application-level mechanisms** for congestion control, jitter management, error recovery, and media adaptation.

Thus, **low latency is generally more important than perfect reliability for real-time audio/video**.

3. Recommend suitable protocol-level or application-level improvements.

The following improvements can enhance conferencing quality:

Protocol-level improvements

- Prefer **UDP-based real-time media transport** instead of TCP where supported.
- Use **RTP/RTCP over UDP** for audio and video.
- Use congestion-control mechanisms suitable for real-time traffic.
- Prioritize voice/video packets using **QoS**.
- Apply traffic shaping to prevent bandwidth congestion.
- Use forward error correction where appropriate to tolerate packet loss.

Application-level improvements

- Implement an **adaptive jitter buffer**.

- Dynamically reduce video resolution and frame rate during congestion.
- Use adaptive bitrate streaming.
- Reduce excessive buffering because it increases end-to-end delay.
- Apply packet-loss concealment for missing audio packets.
- Prioritize audio over video because voice communication is more sensitive to delay.
- Monitor network conditions and dynamically select suitable media quality.

Conclusion

The major issue is the combination of **packet loss, high jitter, transport behavior, and buffering**. A UDP-based real-time transport mechanism combined with adaptive application-level buffering and QoS can significantly improve conferencing quality.

Case Study 2: Global DNS Performance

1. Analyze the reasons behind the high DNS resolution time.

The average DNS lookup time is approximately **800 ms**, which is very high for a web application. Several factors can cause this delay:

- **Geographic distance**
    - If users are located far away from the DNS server, DNS requests and responses experience greater network propagation and transmission delays.
- **Centralized DNS infrastructure**
    - If DNS requests are handled by only a few servers, the servers may become overloaded during peak periods.
- **Multiple DNS queries**
    - A DNS resolution may require communication with:
    - Root DNS servers
    - TLD servers
- Authoritative DNS servers
- Each additional query can increase resolution time.
- **Poor caching**

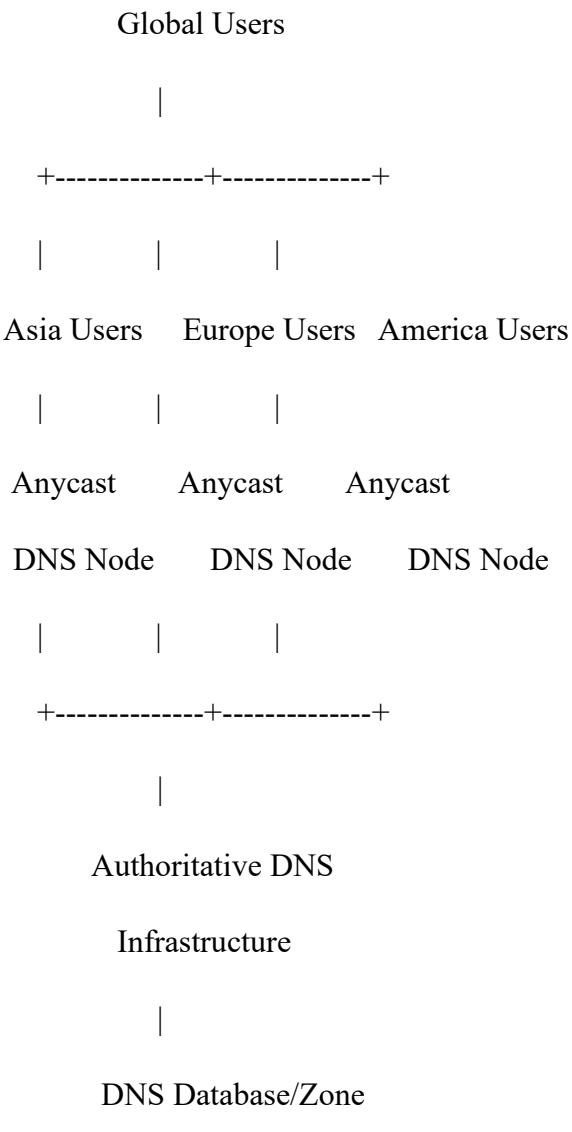
If DNS records are not cached effectively, users repeatedly perform complete DNS resolution.

- **Network congestion**
  Congestion and packet loss can cause DNS requests to be delayed or retransmitted.
- **Inefficient TTL configuration**
  Very short TTL values cause cached DNS records to expire frequently, increasing the number of DNS queries.

2. Propose a suitable DNS architecture using Anycast DNS, DNS pre-fetching, and TTL optimization to reduce lookup time below 50 ms.

A **globally distributed Anycast DNS architecture** should be implemented.

Proposed architecture



Anycast DNS

The same DNS IP address is advertised from multiple geographically distributed DNS servers.

When a user sends a DNS query, Internet routing directs the request to a nearby or optimal DNS node.

For example:

- Asian users → Asian DNS node
- European users → European DNS node
- American users → American DNS node

This reduces network round-trip time.

DNS caching

Resolvers should cache frequently requested records.

For example:

User → Local DNS Cache → Cached IP

If the record exists in the cache, the resolver can respond immediately without contacting authoritative servers.

DNS pre-fetching

Frequently accessed DNS records can be refreshed before their TTL expires. This prevents cache expiration from causing a full DNS lookup at the time the user requests the website.

TTL optimization

TTL should be selected according to the stability of the DNS records.

For relatively stable services, a longer TTL can be used to increase caching and reduce DNS queries.

For frequently changing records, a shorter TTL can be used.

Target

The combination of:

Anycast + caching + pre-fetching + optimized TTL

can substantially reduce DNS lookup latency and help achieve the target of **less than 50 ms for cached/nearby resolutions**, although the exact latency depends on geographic location, resolver behavior, and network conditions.

---

3. Evaluate the advantages and limitations of the proposed architecture.

Aspect	Advantages	Limitations
Performance	Anycast reduces geographic latency	Requires global infrastructure
Caching	Reduces repeated DNS queries	Cached information may become outdated
Pre-fetching	Reduces delays caused by expired records	Generates additional background DNS traffic
TTL optimization	Improves cache efficiency	Incorrect TTL selection can cause stale data
Scalability	Traffic is distributed across multiple DNS nodes	More servers increase management complexity
Availability	Failure of one node can be handled by another	Requires proper routing and monitoring
Fault tolerance	No single DNS server failure affects the entire service	Anycast configuration errors can affect multiple regions

Conclusion

A distributed Anycast DNS architecture with caching, pre-fetching, optimized TTL values, and multiple authoritative servers provides **low latency, high scalability, and high availability**.

Case Study 3: Distributed DNS for Global Cloud Services

1. Examine the impact of centralized DNS architecture on application performance.

The company's centralized DNS architecture creates a **single point of congestion and failure**.

All DNS requests are directed to one primary server.

During promotional events, millions of users generate DNS requests simultaneously. This causes:

- DNS server overload
- Increased queueing delay
- Higher CPU and memory utilization
- Longer DNS response times
- Increased network traffic toward one location
- Potential DNS timeouts
- Poor application startup performance

The reported **800 ms DNS delay** means users must wait significantly longer before their browsers can establish connections to the actual cloud service.

For example:

User

↓

Central DNS Server

↓

800 ms DNS resolution

↓

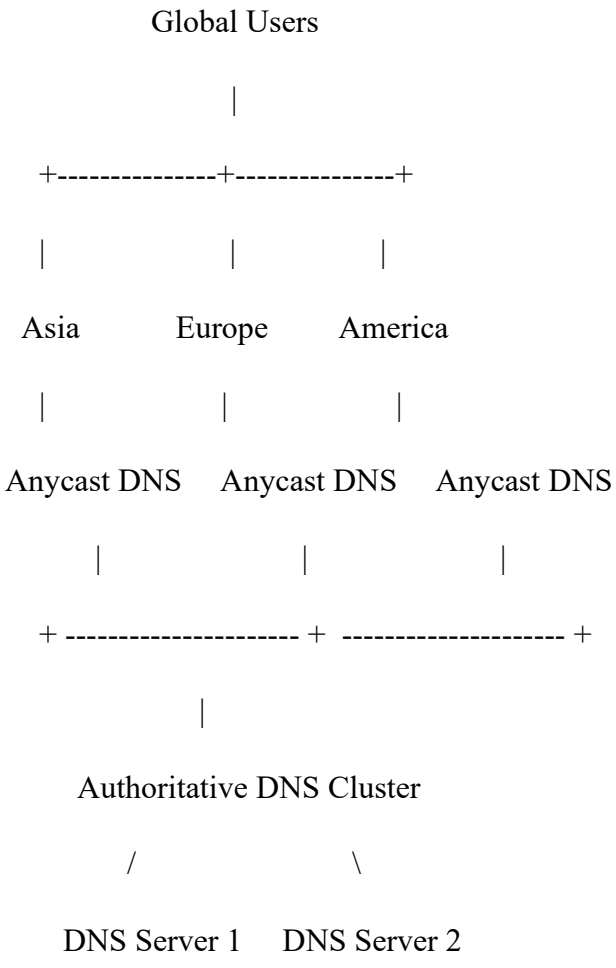
Application Server

DNS delay directly contributes to the total page-load time.

Therefore, centralized DNS architecture is unsuitable for a globally distributed, high-volume cloud service.

2. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

A distributed DNS architecture should use **Anycast DNS servers located in multiple geographic regions**.



Components

1. Anycast DNS

Multiple DNS servers advertise the same IP address.

Users are automatically routed toward an appropriate DNS node.

2. Multiple authoritative DNS servers

At least two or more authoritative servers should be deployed in separate locations.

This prevents a single-server failure from taking down DNS service.

3. Recursive DNS caching

Resolvers cache frequently requested domain records and answer future requests locally.

4. Load distribution

DNS traffic is distributed geographically rather than being processed by one centralized server.

5. Health monitoring

DNS nodes should be continuously monitored. Failed nodes should be removed from routing advertisements.

3. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

TTL

Time To Live (TTL) determines how long DNS information can remain cached.

A higher TTL:

- Reduces repeated DNS queries
- Reduces DNS server load
- Improves response speed
- Improves scalability

However, excessively high TTL values can cause stale DNS information after an IP address changes.

Anycast routing

Anycast allows multiple DNS servers to use the same IP address.

Benefits include:

- Lower latency
- Geographic distribution
- Load distribution

- Better availability
- Automatic routing around failed locations

If one Anycast location fails, routing can direct users to another available DNS node.

DNS caching

Caching stores DNS responses closer to users.

For example:

First request:

User → DNS Resolver → Authoritative DNS

Later requests:

User → DNS Resolver Cache → Immediate response

Caching significantly reduces lookup time and authoritative-server workload.

Overall effect

Technique	**Performance**	**Fault Tolerance**
TTL optimization	Reduces repeated lookups	Depends on appropriate TTL
Anycast	Reduces geographic latency	Provides regional redundancy
DNS caching	Provides very fast responses	Reduces dependency on authoritative servers

Conclusion

The centralized DNS system should be replaced with a **distributed Anycast DNS architecture supported by multiple authoritative servers, effective caching, health monitoring, and optimized TTL values**. This architecture can handle millions of global users more efficiently and prevent DNS from becoming a performance bottleneck.

Case Study 4: High-Frequency Stock Trading Platform

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.

The order acknowledgment latency has increased from **1 ms to 40 ms**. The observed TCP retransmissions, longer connection establishment times, and excessive buffering indicate several TCP-related problems.

A. TCP Flow Control

TCP uses the **receiver window** to prevent the sender from transmitting more data than the receiver can handle.

If the receive buffer becomes full, the receiver advertises a smaller receive window. The sender must then slow down.

In a high-frequency trading system, this can create:

High order rate

↓

Receive buffer fills

↓

Small TCP receive window

↓

Sender slows down

↓

Higher transaction latency

Thus, inefficient receive-buffer sizing can contribute to latency.

B. Nagle's Algorithm

Nagle's algorithm combines small TCP segments into larger segments to improve network efficiency.

It waits for acknowledgment before sending additional small packets in certain situations.

This is useful for reducing network overhead, but it can be harmful in **high-frequency trading**, where orders and acknowledgments may be very small and latency-sensitive.

For example:

Small order

↓

Nagle buffers packet

↓

Wait for ACK / additional data

↓

Packet transmission delayed

↓

Trade acknowledgment delayed

Therefore, Nagle's algorithm can introduce unwanted latency.

C. SYN Queue Limitations

TCP connection establishment uses the three-way handshake:

Client → SYN → Server

Client ← SYN-ACK ← Server

Client → ACK → Server

During market opening, a large number of clients may attempt to establish connections simultaneously.

If the server's SYN queue is too small, it may become full.

This can cause:

- Connection establishment delays
- Dropped SYN requests

- Retransmissions
- Longer connection setup time
- Increased order-processing latency

2. Explain how each factor affects transaction processing in high-frequency systems.

TCP Factor	Effect on Trading System
Flow control	Limits sending rate when receiver buffers are full
Nagle's algorithm	Buffers small packets and introduces transmission delay
SYN queue limitations	Delays or rejects new TCP connections
Retransmissions	Increase latency and consume network resources
Excessive buffering	Creates queueing delay and increases response time

Flow control

High-frequency trading generates many transactions within a short period. If buffers are insufficient, flow control can restrict transmission and increase queueing.

Nagle's algorithm

Trading orders are usually small and latency-sensitive. Waiting to combine packets can increase order-to-acknowledgment time.

SYN queue

If many connections are created at market opening, an insufficient SYN backlog can delay connection establishment before the actual transaction even begins.

Therefore, all three mechanisms can contribute to the observed increase from **1 ms to 40 ms**.

3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

- **Disable Nagle's algorithm for latency-sensitive connections**
- Use:
    - TCP_NODELAY
    - This allows small packets to be transmitted immediately instead of waiting for aggregation.
    - This is particularly suitable for small, latency-sensitive trading messages.

- **Optimize TCP receive and send buffers**
    - Configure appropriate TCP buffer sizes based on traffic volume.
    - Avoid:
    - Buffers that are too small → unnecessary flow-control blocking
    - Buffers that are excessively large → excessive queueing and latency

- **Increase SYN backlog**
    - Increase the server's SYN queue/backlog capacity to handle sudden connection bursts.
    - This helps prevent connection establishment failures during market opening.

- **Use persistent TCP connections**
    - Instead of establishing a new TCP connection for every order:
    - Bad:
    - Order → TCP connection → Close
    - Order → TCP connection → Close
    - Use:

      Persistent TCP connection

      ↓

      Order 1

      Order 2

      Order 3

      Order 4...

This eliminates repeated three-way handshakes and reduces latency.

- **Monitor retransmissions**
  - Investigate packet loss, congestion, and network errors causing TCP retransmissions.
  - Reducing packet loss can significantly improve transaction reliability.

- **Use low-latency network configuration**
  - The organization can also use:
  - Dedicated network links
  - QoS for trading traffic
  - Low-latency network interfaces
  - Proper congestion management
  - Server load balancing
  - Connection pooling where appropriate

Overall Conclusion

The stock trading platform's latency increase is likely caused by a combination of **TCP flow-control restrictions, Nagle's algorithm, SYN queue limitations, retransmissions, and excessive buffering**.

The most appropriate improvements are:

1. Disable Nagle's algorithm using TCP_NODELAY
2. Optimize TCP send/receive buffers
3. Increase SYN backlog capacity
4. Use persistent TCP connections
5. Reduce packet loss and retransmissions
6. Apply appropriate QoS and low-latency networking

These changes can reduce unnecessary TCP delays while maintaining reliable delivery of financial transactions.